

The Legibility Dividend

An Executive Model of Verifiable Software Production

Measured evidence from an instrumented, agent-operated engineering system

The Flux Foundation

model computed by `flux-axiv-latex`, thermodynamic floor by `flux-science`,
related work drawn from a live arXiv sweep

July 26, 2026

216×

MEASURED — inner-
loop latency collapse
(215.9s → 1.0s)

\$59,820

DERIVED — recov-
ered engineer-cost
per head, per year

37×

DERIVED — return on
the four-part mea-
surement portfolio

Abstract

Software organisations are asked to fund verification — tests, provenance, measurement, post-mortems — out of a budget that rewards features. The usual defence is moral (“quality matters”), and it loses. This paper makes the financial case instead, and it makes it from a real instrumented record rather than a survey: a small, fully measured engineering system whose build system, test instruments, incident taxonomy and artifact provenance were all quantified over three months. Every figure below is *computed at document-generation time* from those measurements plus explicitly declared cost assumptions; nothing is quoted from a vendor deck. Four results. (1) A single build-system defect — a fingerprint file that cache-restored units never wrote — inflated “no-op” rebuilds to **215.9s**; repairing it (216×) is worth **\$59,820** per engineer per year at the stated rates, paying back in **14.6** working days. (2) A test suite is an information channel, and one measured suite carried **105** passing tests while an independent probe found **0.4%** of reads returning data — a *Verification Information Yield* of **0.0014 bits**, i.e. a green build that meant nothing. Restoring an independent correctness probe buys **0.20 bits** at **\$234,433** of avoided expected loss per bit. (3) Seventy-one real incidents cluster into ten classes; three classes cover **43.7%** of them, which turns a qualitative “improve quality” mandate into a ranked budget. (4) Against the Landauer floor, the work of deciding whether anything changed costs 5.76×10^{18} × the thermodynamic minimum even *after* the repair: essentially all engineering cost is coordination, not physics, and the savings pool is nowhere near exhausted. We close with a decision rule (the Legibility Ratio), a 90-day program, and an explicit list of what remains assumed rather than measured.

1 Executive summary

This section is the whole paper for a reader with four minutes. Each finding names its evidence grade: ● MEASURED means taken from an instrument on a real system, ⊕ DERIVED means computed in this document from measured inputs, △ ASSUMED means a declared assumption a reader may replace.

Findings

1. **The inner loop was the largest single cost centre, and nobody had priced it.** • MEASURED An unchanged crate took 215.9s to confirm it was unchanged; the heaviest binary took 359 s across 166 dirty units. Root cause: 19 poisoned fingerprint roots cascading into 147 downstream units. After repair: 1.0s no-op, 8.9s for a three-crate edit, 36% unit cache hit rate.
2. **A green test suite can carry zero information.** • MEASURED 105 passing tests coexisted with 0.4% of storage reads returning data. ⊕ DERIVED Verification Information Yield = 0.0014 bits. A suite that passes whether or not the system works is not a control; it is a cost.
3. **Fast components beside slow systems are bug reports, not victories.** • MEASURED A component measured 12500× faster than the live system it served — the gap lay in code nobody had benchmarked.
4. **Failures are not random; they are a short list.** • MEASURED 71 incidents, ten classes, top three = 43.7%, top five = 64.8%. 7 of the 71 were the measuring instrument itself being wrong.
5. **Provenance is an audit asset with a computable yield.** • MEASURED A constant 2568-byte proof verifies a whole history in 342ms, and an incremental check costs 3 μs — roughly 105,263 verifications per dollar of machine time.
6. **Physics is not the constraint.** ⊕ DERIVED Post-repair, answering “did anything change?” still costs $5.76 \times 10^{18} \times$ the Landauer floor; 100.0% of the cost is coordination. There are 62 further doublings of efficiency available before thermodynamics objects.

Three decisions this paper supports

1. **Fund the inner loop before the feature list.** At the stated rates the latency repair returns 16× its cost and pays back in 14.6 working days — the highest-yield line item in the portfolio, and the one no roadmap contains.
2. **Require an independent correctness probe beside every performance or health claim.** Cost ceiling: 500 engineer-hours per release still breaks even (\$47,500 of avoided expected loss per release at the stated escape cost).
3. **Make evidence grades a reporting standard.** Every number in a status report carries MEASURED, DERIVED or ASSUMED. This is free, and it is the control that would have caught 23% of the incident record (= the measurement and stale-state classes).

2 Why legibility is a financial instrument

The literature on software cost is unambiguous that poor quality is expensive at civilisational scale [13], and equally unambiguous that the mechanisms which would reduce it are under-adopted. Two examples frame this paper. First, build performance: a large-scale study of 513,384 builds across 1,279 projects found that only 30% adopt caching at all [2], and a case study of a flagship infrastructure project documents the cost of *downgrading* a build system as a deliberate trade [1]. Second, verification signal: an industrial study of a major database system reports flaky tests giving “an ambiguous signal about the quality of the code” and

interfering with automated assessment [8], a finding replicated across languages and resource configurations [9, 10].

Both are usually discussed as engineering hygiene. They are better understood as *unpriced liabilities*. The reason they stay unpriced is structural: an organisation cannot put a number on a cost it does not measure, and the measurement itself competes for the same budget. That circularity — you must spend to learn what spending would save — is precisely what a business case has to break. Technical-debt research has reached the same conclusion from the other direction: prioritisation fails when technical and business framings are misaligned [20], management tooling stays unadopted because its time and cost are perceived as high [21], and the effort penalty of unpaid debt is measurable when anyone bothers to measure it [19].

This paper contributes the missing artifact: a worked, reproducible valuation from a system where the measurements actually exist. The system is small and unusual — one operator, a fleet of automated agents, a public chain and its build orchestrator — and §11 is explicit about what that costs in generalisability. What it buys, in exchange, is a record where every claim was already graded before this paper existed.

3 The instrument: evidence grades as a management control

The engineering record this paper draws on carries two standing rules. They are cheap, they are unusual, and they are the reason the numbers below are worth anything.

Rule 0 — a fast component beside a slow system is a bug report, not a victory. Every claim states which rung of the ladder it sits on: microbenchmark, in-process harness, or live system. The canonical violation in the record is a store measured at ten million records per second while the live system it served ran at eight hundred blocks per second — four orders of magnitude apart, with the true bottleneck in a component nobody had benchmarked.

Rule 1 — grade the claim, not the claimant. Each number carries an evidence grade. This paper uses three: MEASURED (an instrument on a real system), DERIVED (computed here from measured inputs and declared assumptions), ASSUMED (a stated input a reader may replace). The full record uses seven, adding simulated, staged, live and conjectural.

The reason a board should care is that the record also contains a chapter on *instruments that lied* — four documented cases where the measuring tool reported success without doing the work: a combined test tool reporting “0 passed, 0 failed” when the test binary failed to compile (readable as “no failures”); a stale-binary selection that measured three-week-old code; silently dropped test filters, so an expensive benchmark that appeared to run in 0s had not run; and a cache reporting its own hit rate as victory while an independent probe measured 0.4% of reads returning data. Seven of the 71 recorded incidents (9.9%) are of this kind. *An organisation’s dashboard is part of its attack surface.*

The one-page reporting standard

Every quantitative line in a status report or board pack carries: (a) the number; (b) its evidence grade; (c) the instrument that produced it; (d) the independent check that would falsify it. Cost: zero incremental spend, one column. Expected effect on this record: the measurement and stale-state classes together are 23% of all incidents (16 of 71).

4 Dividend 1 — the inner loop is a wage bill

The measurement. • MEASURED Builds that should have been no-ops were not. Confirming that an unchanged crate was unchanged cost 215.9s; the heaviest binary cost 359s across 166

units that were re-declared dirty on every invocation. The cause was not compilation but bookkeeping: units restored from the content cache skipped the compiler, so the fingerprint dependency file was never written, and the build tool marked 19 roots permanently stale, cascading into 147 downstream units. After the repair — capture, require and materialise that file on cache restore — the same no-op measured 1.0s (heal run 0.56s), a three-crate incremental edit including the heavy binary measured 8.9s, and the unit-level cache hit rate settled at 36%. Two long-held beliefs died in the same measurement: that a compiler-wrapper environment variable was fragmenting build identity, and that the shared cache was empty (a symlink that a disk-usage probe reported as zero, with 17 GB behind it).

The model. $\triangle$ ASSUMED An engineer costs \$95 per hour fully loaded and works 230 days a year. $\triangle$ ASSUMED They invoke the inner loop 40 times a day. $\triangle$ ASSUMED A wait longer than 10s triggers a task switch with probability 0.35, and each switch costs 90s of recovery. $\oplus$ DERIVED Direct saving: $40 \text{ invocations} \times 214.9 \text{ s} = 2.39 \text{ h/day}$. Switch saving: 0.35 h/day . Total $2.74 \text{ h/day} = 630 \text{ h/engineer/year} = \$59,820 = 0.34 \text{ recovered full-time equivalents per engineer}$.

Engineers	Hours/year recovered	Cost recovered	FTE equivalent
1	630	\$59,820	0.3
10	6,297	\$598,204	3.4
100	62,969	\$5,982,044	34.2
1,000	629,689	\$59,820,444	342.2

The linearity is an assumption, not a measurement: at large team sizes shared build infrastructure contends, which is exactly the trade the Kubernetes build-system study documents [1], and cache adoption is itself the exception rather than the rule [2]. Read the 1000-engineer row as an upper bound on a real organisation.

Sensitivity. The single most contestable input is invocation frequency. $\oplus$ DERIVED

Invocations/day	Hours/day	Hours/year	Cost/year (1 engineer)
10	0.68	157	\$14,955
20	1.37	315	\$29,910
40	2.74	630	\$59,820
80	5.48	1,259	\$119,641

Even the conservative row — ten invocations a day — recovers more than the cost of the investigation that found the defect (40 engineer-hours, \$3,800). $\oplus$ DERIVED Payback: 14.6 working days. Legibility Ratio: $16\times$. The machine-energy side is real but small at this scale: 395 kWh and \$119 per engineer per year.

5 Dividend 2 — Verification Information Yield

The second dividend needs a new instrument, because the industry standard one — “the suite is green” — is not a measurement. It is a claim about a claim.

The measurement. $\bullet$ MEASURED A storage layer in this record passed 105 of 105 unit tests for its entire service life while destroying data at scale. An independent probe — one that asserted the value came back rather than timing the call — measured 0.4% of reads returning data. The suite was not lying about its own execution; it was answering a different question than anyone believed.

The instrument. Treat a test run as a communication channel. Let D be the event that a release contains a defect of the class in question and G the event that the suite reports green. The suite’s value is the mutual information

$$\text{VIY} = I(D; G) = \sum_{d \in \{0,1\}} \sum_{g \in \{0,1\}} P(d, g) \log_2 \frac{P(d, g)}{P(d)P(g)} \quad \text{bits per run.}$$

$\triangle$ ASSUMED With $P(D) = 0.05$ per release, the measured false-green case has $P(G | D) = 1.00$ — the suite passes even when the defect is present — and $P(G | \neg D) = 0.98$ on a clean tree. $\oplus$ DERIVED Then $\text{VIY} = 0.0014$ bits — over two orders of magnitude below what a working probe yields, and indistinguishable from zero (the residual is an artifact of the assumed clean-tree flake rate, which perversely makes green *slightly more* likely when the defect is present). The green build carried *no usable information* about the property anyone cared about. Adding an independent correctness probe that catches the defect class with $P(G | D) = 0.05$ lifts the yield to **0.2040 bits** per run.

The money. $\triangle$ ASSUMED An escaped defect of this class costs \$1,000,000; there are 12 releases a year. $\oplus$ DERIVED Expected loss before: $P(D) \times P(G|D) \times C = \$50,000$ per release. After: \$2,500. The probe therefore buys \$47,500 per release, \$570,000 per year, at a cost of 80 engineer-hours per year (\$7,600). Legibility Ratio 75 $\times$; payback 4.9 days. Two framings a board can act on:

- **Break-even budget:** up to **500 engineer-hours per release** may be spent on independent verification of this one property before it stops paying.
- **Price per bit:** the first 0.204 bits of verification information are worth **\$234,433** each. Information, not effort, is the unit that prices correctly.

The flaky-test literature is the same problem seen from the noise side: non-deterministic tests degrade the channel until teams stop trusting it [9], industrial studies tie the ambiguity directly to slowed release assessment [8], resource contention alone moves failure rates [10], reproduction is hard enough to need dedicated datasets [12], and repair is now an automation target at industrial scale [11]. VIY makes the shared quantity explicit: flakiness and false-green are both *information losses*, and both should be funded against the same number.

6 Dividend 3 — the failure taxonomy as a budget allocator

The measurement. $\bullet$ MEASURED 71 distinct failures from one system’s own record, each carrying its file, its number and the height at which it was found; severity 15 catastrophic, 44 major, 12 moderate. They fall into ten classes, and the distribution is steep.

Class	#	Share	Cumulative
Silent divergence	12	16.9%	16.9%
Protocol wedge	10	14.1%	31.0%
Stale state	9	12.7%	43.7%
Resource exhaustion	8	11.3%	54.9%
Bad default	7	9.9%	64.8%
Measurement (the instrument lied)	7	9.9%	74.6%
Economic (value created/destroyed)	6	8.5%	83.1%
Key management	6	8.5%	91.5%
Toolchain	5	7.0%	98.6%
Serialization panic	1	1.4%	100.0%

Why an executive should read a failure table. $\oplus$ DERIVED Three classes cover 43.7% of all incidents and five cover 64.8%. That converts an unfundable mandate (“improve quality”) into a ranked allocation with named owners. 21.1% of the record was catastrophic — funds, consensus or fleet at stake — so the tail is not academic. And the single most uncomfortable

row is *measurement* at 7 incidents (9.9%): one failure in ten was the organisation’s own instrument.

△ ASSUMED Credit the taxonomy with preventing just *one* catastrophic repeat every three years — 0.33 per year, against a record that already contains fifteen — and at \$1,000,000 per catastrophic escape it is worth **\$333,333** a year against an authoring cost of 60 engineer-hours (\$5,700): a Legibility Ratio of 58×. The assumption is the rate; the measurement is the distribution. A reader who believes only half the effect still sees 29×.

The mechanism is not moral either. A public taxonomy makes a class of failure *recognisable before it is catastrophic*, and in this record the majority of classes produced a structural guard — an invariant, a refusal, a changed default — rather than a patch. That is the conversion worth funding: incidents into shape.

7 Dividend 4 — provenance as an audit asset

The measurement. • MEASURED Artifacts in this system carry signed provenance, and the system’s history carries a constant-size proof: 2568 bytes regardless of chain length, full verification in 342 ms at one hundred thousand blocks, and an incremental check against an already-verified state in 3 μs with zero data downloaded. ⊕ DERIVED At the stated machine-time proxy that is roughly 105,263 full verifications per dollar.

Why this is a business asset and not a security hobby. The supply-chain literature is clear that provenance frameworks work and equally clear that adoption stalls on cost: a study of SLSA deployment finds interest without uptake [15]; an analysis of software signing states plainly that it “imposes tooling and operational costs to implement in practice” [16]; the reproducible-build tradition demonstrates the end-to-end version is achievable [17], and hardware-rooted variants push the trust boundary further [18]. Meanwhile the build pipeline itself is now a first-class attack surface [6]. The adoption gap is therefore a *pricing* gap, and pricing is a board’s job.

△ ASSUMED Signed artifacts plus written commitment records displace 200 external audit hours a year at \$150 per hour, and address half of an annual supply-chain incident risk of 2% at \$5,000,000. ⊕ DERIVED Value \$80,000 against 120 engineer-hours (\$11,400); Legibility Ratio 7×. The defensible core of that claim is the cheap-verification measurement: when a check costs 3 μs, “verify continuously” stops being a slogan and becomes a line item that rounds to zero.

8 The floor: how much of engineering cost is physics?

One number keeps executives honest about how much headroom remains. Erasing a bit at temperature T costs at least $k_B T \ln 2$; at $T = 300$ K that is 2.87×10^{-21} J. Deciding whether a workspace of 170 units has changed requires, at minimum, comparing one digest per unit — 43520 bits — so thermodynamics prices the entire question at $E_{\min} = 1.25 \times 10^{-16}$ J.

⊕ DERIVED Before the repair, that question consumed 215.9 s on a 48-core machine drawing 720 W = 1.55×10^5 J: a factor of 1.24×10^{21} above the floor. After the repair it consumes 720 J — still 5.76×10^{18} above the floor, i.e. 100.0000% of the energy is coordination overhead and 62 further doublings of efficiency remain physically available. *The constraint is never the universe; it is agreement.* That is the strategic point of the whole paper: a cost that is 100.0000% organisational is a cost that responds to organisational instruments — measurement, grading, provenance — and not to hardware procurement.

The floor itself is not folklore: experimental work approaches $k_B T \ln 2$ in real devices [26], and adiabatic logic families are explicitly engineered against it [25]. We cite them to fix the denominator, not to suggest anyone should build a build system out of superconductors.

9 The decision rule: the Legibility Ratio

Collecting the four dividends gives a single, auditable decision rule. For any proposed act of measurement m — an instrument, a probe, a taxonomy, a signature — define

$$\mathcal{L}(m) = \frac{V_{\text{avoided}}(m)}{C_{\text{measure}}(m)}, \quad \text{fund } m \iff \mathcal{L}(m) > 1 \text{ and the inputs to } V \text{ are graded.}$$

Dividend	Annual value	Annual cost	$\mathcal{L}$	Payback (days)
D1 — build latency	\$59,820	\$3,800	16×	14.6
D2 — verification yield	\$570,000	\$7,600	75×	4.9
D3 — failure taxonomy	\$333,333	\$5,700	58×	—
D4 — provenance/audit	\$80,000	\$11,400	7×	—
Portfolio	\$1,043,154	\$28,500	37×	10.0

⊕ DERIVED The portfolio is 300 engineer-hours (\$28,500) returning \$1,043,154 — 37×, payback 10.0 days. The second column is the honest one to attack: three of the four values contain an assumed severity or probability, and §11 lists each. The first column does not: those are instrument readings.

The rule stated for a sceptic

Halve every value in the table and double every cost. The portfolio still returns 9×. A conclusion that survives a factor of four against itself is a decision, not a forecast.

10 A 90-day program, with measurement gates

Each item names what would prove it worked. An item without a falsifiable gate is not in the program.

Window	Action and measurement gate	Grade at start
Days 1–15	Instrument the inner loop. Record the latency distribution of the no-change build across the fleet. Gate: a published histogram, and the 90th percentile named. If the 90th percentile exceeds 10 s, the defect class in §4 is present.	△ ASSUMED
Days 1–30	Pair every health claim with an independent probe. For each dashboard that reports its own success, add one external assertion of the property. Gate: VIY > 0 demonstrated by a deliberately broken build that the suite must fail.	△ ASSUMED
Days 15–45	Adopt evidence grades in reporting. One extra column in every status number. Gate: a board pack in which every figure carries a grade and an instrument.	△ ASSUMED
Days 30–60	Write the taxonomy. Classify the last two years of incidents; publish internally. Gate: three classes covering ≥43% of incidents, each with a named owner and one structural guard shipped.	△ ASSUMED
Days 45–90	Sign the artifacts. Provenance on every released binary plus a written commitment record for every external promise. Gate: an auditor reproduces one release end-to-end without asking the team a question.	△ ASSUMED

The ordering is deliberate: latency first because it funds the rest, probes second because they protect every subsequent measurement, grading third because it is free, taxonomy fourth because it needs the previous three to be trustworthy, provenance last because it is the only item whose value is realised outside the organisation.

11 Threats to validity, and what is still assumed

This paper’s own standard forbids ending on the strong numbers. The following are the reasons a careful reader should discount them, stated before anyone else has to find them.

1. **$N = 1$, and the one is unusual.** The measured system is a single engineering household: one operator, a fleet of automated agents on shared infrastructure, one public chain and its build orchestrator. It is not a sample of industry. Its incident record is unusually complete *because* it is small. The distribution of failure classes should be re-measured, not imported.
2. **Every dollar is derived, none is invoiced.** No figure in this paper came from an accounting system. Labour is priced at an assumed \$95 per hour, escapes at an assumed \$1,000,000, supply-chain incidents at an assumed \$5,000,000 with an assumed 2% annual probability. Replace them; the model is a function, not a result.
3. **Linear team scaling is wrong at the top of the table.** The 1000-engineer row ignores build-infrastructure contention, which the literature documents as a real trade [1], and assumes cache adoption that 70% of projects do not have [2].
4. **The latency win was a defect repair, not a technology.** $215.9\text{ s} \rightarrow 1.0\text{ s}$ is the removal of a specific pathology (unwritten fingerprint files on cache restore). A team without that pathology should expect a far smaller number, and the honest residuals were recorded with the win: a version bump legitimately mints new unit identities, so the first one or two runs after a release are slow by construction.
5. **VIY’s probabilities are priors, not frequencies.** $P(D)$ and $P(G|D)$ after the probe are assumed. The *before* case is not: $P(G|D) = 1.00$ was observed — a full suite green against a live data-loss defect.
6. **Agent-operated engineering is early.** The record was produced by a human operator directing automated agents, a regime whose behaviour is actively under study [22, 23, 24], and whose internal work-logs are trust-internal rather than third-party verifiable. Treat process claims accordingly; treat the instrument readings as instrument readings.
7. **What this paper does not claim.** No claim that measurement substitutes for engineering judgement; no claim that the four dividends compose additively in a large organisation (they are reported separately for that reason); no claim of causality between publishing a taxonomy and the subsequent incident rate — that gate is stated in §10 precisely because it has not yet been passed.

12 Related work

Retrieved by a live arXiv query at generation time; each entry is glossed with the opening claim of its own abstract, unedited.

- [1] *The Cost of Downgrading Build Systems: A Case Study of Kubernetes* — Since developers invoke the build system frequently, its performance can impact productivity.
- [2] *The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds* — Continuous Integration provides early feedback by automatically building software, but long build durations can hinder developer productivity.
- [3] *A model and framework for reliable build systems* — Reliable and fast builds are essential for rapid turnaround during development and testing.

- [4] *Built to Last or Built Too Fast? Evaluating Prediction Models for Build Times* — Automated builds are integral to Continuous Integration.
- [5] *Practitioners’ Challenges and Perceptions of CI Build Failure Predictions at Atlassian* — Continuous Integration build failures could significantly impact the software development process and teams, such as delaying the release of new features and reducing developers’ productivity.
- [6] *Build Code is Still Code: Finding the Antidote for Pipeline Poisoning* — Open source C code underpins society’s computing infrastructure.
- [7] *Quantifying and Characterizing Clones of Self-Admitted Technical Debt in Build Systems* — Self-Admitted Technical Debt annotates development decisions that intentionally exchange long-term software artifact quality for short-term goals.
- [8] *Flaky Tests in a Large Industrial Database Management System: An Empirical Study of Fixed Issue Reports for SAP HANA* — Flaky tests yield different results when executed multiple times for the same version of the source code.
- [9] *Predicting Flaky Tests Categories using Few-Shot Learning* — Flaky tests are tests that yield different outcomes when run on the same version of a program.
- [10] *The Effects of Computational Resources on Flaky Tests* — Flaky tests are tests that nondeterministically pass and fail in unchanged code.
- [11] *FlakyGuard: Automatically Fixing Flaky Tests at Industry Scale* — Flaky tests that non-deterministically pass or fail waste developer time and slow release cycles.
- [12] *A Dataset of Reproducible Flaky-Test Failures* — Flaky tests pass and fail non-deterministically when run on the same version of code.
- [13] *Software is infrastructure: failures, successes, costs, and the case for formal verification* — In this chapter we outline the role that software has in modern society, along with the staggering costs of poor software quality.
- [14] *Formal Verification of PLCs as a Service: A CERN-GSI Safety-Critical Case Study* — Increased technological complexity and demand for software reliability require organizations to formally design and verify safety-critical programs.
- [15] *Analyzing Challenges in Deployment of the SLSA Framework for Software Supply Chain Security* — In 2023, Sonatype reported a 200% increase in software supply chain attacks, including major build infrastructure attacks.
- [16] *Why Software Signing (Still) Matters: Trust Boundaries in the Software Supply Chain* — Software signing provides a formal mechanism for provenance by ensuring artifact integrity and verifying producer identity.
- [17] *Building a Secure Software Supply Chain with GNU Guix* — The software supply chain designates the series of steps taken to go from source code published by developers to executables running on users’ computers.
- [18] *Hardware-Enforced Integrity and Provenance for Distributed Code Deployments* — Deployed microservices must adhere to a multitude of application-level security requirements and regulatory constraints imposed by mutually distrusting application principals.
- [19] *Measuring the Impact of Technical Debt on Development Effort in Software Projects* — Technical debt refers to the trade-offs between code quality and faster delivery, impacting future development with increased complexity, bugs and costs.

- [20] *Business-Driven Technical Debt Prioritization* — Technical debt happens when teams take shortcuts on software development to gain short-term benefits at the cost of making future changes more expensive.
- [21] *Automating Technical Debt Management: Insights from Practitioner Discussions in Stack Exchange* — Managing technical debt is essential for maintaining long-term software projects.
- [22] *Unified Software Engineering Agent as AI Software Engineer* — The growth of Large Language Model technology has raised expectations for automated coding.
- [23] *Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories* — LLM-based agents are increasingly employed to automate complex software engineering tasks, such as program repair and issue resolution.
- [24] *Process-Level Trajectory Evaluation for Environment Configuration in Software Engineering Agents* — LLM-based agents show promise for software engineering, but environment configuration remains a bottleneck due to heavy manual effort and scarce large-scale, high-quality datasets.
- [25] *Landauer-Limited Dissipation in Quantum-Flux-Parametron Logic* — The Landauer limit is to irreversible logic what the Carnot cycle is to heat engines.
- [26] *Reset and switch protocols at Landauer limit in a graphene buckled ribbon* — Heat produced during a reset operation is meant to show a fundamental bound known as the Landauer limit.

13 Reproducibility

This document is the output of a Rust binary in the Flux tree (`flux-arxiv-latex/src/bin/legibility_dividend.rs`), built and run through the `fluxc` orchestrator rather than a hand-written `.tex`. Every figure above is computed at generation time from two declared sources: measured constants transcribed from the engineering record (each cited in place with its instrument), and the business assumptions listed in §11, which appear once, at the top of the program, as named variables. The thermodynamic floor comes from `flux-science`’s CODATA Boltzmann constant ($k_B = 1.38 \times 10^{-23}$ J/K); the bibliography and related-work section are generated from a live arXiv API sweep parsed by the same crate. Change an assumption, recompile, and the paper corrects itself — including this sentence’s neighbours. We consider that the only honest way to publish a business case: a valuation you cannot re-derive is a forecast wearing a suit.

14 Coda: legibility before scale

The temptation, having computed a large return, is to claim a large discovery. The record does not support that and the discipline of this corpus forbids it. What was discovered is smaller and more useful: that an engineering organisation which writes down what it measured, on what instrument, and what would prove it wrong, ends up with a balance sheet where before it had opinions. The dividends in this paper are not the reward for building something clever. They are the reward for being *legible* to oneself — and legibility, unlike scale, is available on Monday morning, at the cost of one extra column.

That is also why the uncomfortable rows were kept. A record in which one failure in ten is the instrument’s own fault is not a weak record; it is the only kind that can be trusted about the other nine. An executive reading a report with no such row is not reading a better system. They are reading a less honest instrument.

Sources — the record this paper prices

- *The Idle Machine: What the World Spends Rebuilding What It Already Built*, v0 (2026-07-26) — the sequel: the same measured kernel extrapolated to planetary scale in graded rungs, with the climate headline deliberately deflated and the Jevons rebound applied to its own result. https://quillon.xyz/downloads/FLUX_IDLE_MACHINE_v0.pdf.
- *The Measurement Book: Laws of the SIGIL Graph — the measured record*, v0 (2026-07-25) — the instrument readings used in §4, §5 and §7, including ARC-15 (the build-latency arc), Rule 0, and the calibration chapter on instruments that lied. https://quillon.xyz/downloads/SIGIL_MEASUREMENT_BOOK_v0.pdf.
- *The SIGIL Failure Atlas*, v0 (2026-07-23) — the 71-incident, ten-class taxonomy and severity split tabulated in §6. https://quillon.xyz/downloads/SIGIL_FAILURE_ATLAS_v0.pdf.
- *What the Chain Knows — The Philosophy of the SIGIL Graph*, v0.3 (2026-07-22) — the evidence-grading discipline this paper repurposes as a management control in §3. https://quillon.xyz/downloads/SIGIL_PHILOSOPHY_v0.pdf.
- *The Builder's Chronicle*, v0 (2026-07-23) — development settled on a public ledger; the labour-cost side of the same record. https://quillon.xyz/downloads/SIGIL_BUILDERS_CHRONICLE_v0.pdf.
- *The Adversary's Companion*, v0 (2026-07-24) — the threat model behind §7's provenance valuation. https://quillon.xyz/downloads/SIGIL_ADVERSARYS_COMPANION_v0.pdf.
- *SIGIL — The Variational Chain*, whitepaper v1.3 (2026-07-22) — the system whose operation produced every measurement above. https://quillon.xyz/downloads/SIGIL_WHITEPAPER_v1.pdf.

References

- [1] Gareema Ranjan, Mahmoud Alfadel, Gengyi Sun, et al.: *The Cost of Downgrading Build Systems: A Case Study of Kubernetes*. arXiv:2510.20041 (2025). <https://arxiv.org/abs/2510.20041>
- [2] Taher A. Ghaleb, Daniel Alencar da Costa, Ying Zou: *The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds*. arXiv:2601.19146 (2026). <https://arxiv.org/abs/2601.19146>
- [3] Derrick Coetzee, Anand Bhaskar, George Necula: *A model and framework for reliable build systems*. arXiv:1203.2704 (2012). <https://arxiv.org/abs/1203.2704>
- [4] Ekaba Bisong, Eric Tran, Olga Baysal: *Built to Last or Built Too Fast? Evaluating Prediction Models for Build Times*. arXiv:1712.06796 (2017). <https://arxiv.org/abs/1712.06796>
- [5] Yang Hong, Chakkrit Tantithamthavorn, Jirat Pasuksmit, et al.: *Practitioners' Challenges and Perceptions of CI Build Failure Predictions at Atlassian*. arXiv:2402.09651 (2024). <https://arxiv.org/abs/2402.09651>
- [6] Brent Pappas, Paul Gazzillo: *Build Code is Still Code: Finding the Antidote for Pipeline Poisoning*. arXiv:2601.08995 (2026). <https://arxiv.org/abs/2601.08995>
- [7] Tao Xiao, Zhili Zeng, Dong Wang, et al.: *Quantifying and Characterizing Clones of Self-Admitted Technical Debt in Build Systems*. arXiv:2402.08920 (2024). <https://arxiv.org/abs/2402.08920>
- [8] Alexander Berndt, Thomas Bach, Sebastian Baltes: *Flaky Tests in a Large Industrial Database Management System: An Empirical Study of Fixed Issue Reports for SAP HANA*. arXiv:2602.03556 (2026). <https://arxiv.org/abs/2602.03556>
- [9] Amal Akli, Guillaume Haben, Sarra Habchi, et al.: *Predicting Flaky Tests Categories using Few-Shot Learning*. arXiv:2208.14799 (2022). <https://arxiv.org/abs/2208.14799>
- [10] Denini Silva, Martin Gruber, Satyajit Gokhale, et al.: *The Effects of Computational Resources on Flaky Tests*. arXiv:2310.12132 (2023). <https://arxiv.org/abs/2310.12132>
- [11] Chengpeng Li, Farnaz Behrang, August Shi, et al.: *FlakyGuard: Automatically Fixing Flaky Tests at Industry Scale*. arXiv:2511.14002 (2025). <https://arxiv.org/abs/2511.14002>

- [12] Suzzana Rafi, Mahbub-Ul-Hoque Sumon, Md Erfan, et al.: *A Dataset of Reproducible Flaky-Test Failures*. arXiv:2605.21677 (2026). <https://arxiv.org/abs/2605.21677>
- [13] Giovanni Bernardi, Adrian Francalanza, Marco Peressotti, et al.: *Software is infrastructure: failures, successes, costs, and the case for formal verification*. arXiv:2506.13821 (2025). <https://arxiv.org/abs/2506.13821>
- [14] Ignacio D. Lopez-Miguel, Borja Fernandez Adiego, Matias Salinas, et al.: *Formal Verification of PLCs as a Service: A CERN-GSI Safety-Critical Case Study*. arXiv:2502.19150 (2025). <https://arxiv.org/abs/2502.19150>
- [15] Mahzabin Tamanna, Sivana Hamer, Mindy Tran, et al.: *Analyzing Challenges in Deployment of the SLSA Framework for Software Supply Chain Security*. arXiv:2409.05014 (2024). <https://arxiv.org/abs/2409.05014>
- [16] Kelechi G. Kalu, James C. Davis: *Why Software Signing (Still) Matters: Trust Boundaries in the Software Supply Chain*. arXiv:2510.04964 (2025). <https://arxiv.org/abs/2510.04964>
- [17] Ludovic Courtes: *Building a Secure Software Supply Chain with GNU Guix*. arXiv:2206.14606 (2022). <https://arxiv.org/abs/2206.14606>
- [18] Marcela S. Melara, Mic Bowman: *Hardware-Enforced Integrity and Provenance for Distributed Code Deployments*. arXiv:2106.09843 (2021). <https://arxiv.org/abs/2106.09843>
- [19] Kartik Gupta: *Measuring the Impact of Technical Debt on Development Effort in Software Projects*. arXiv:2502.16277 (2025). <https://arxiv.org/abs/2502.16277>
- [20] Rodrigo Reboucas de Almeida: *Business-Driven Technical Debt Prioritization*. arXiv:1908.01347 (2019). <https://arxiv.org/abs/1908.01347>
- [21] Joao Paulo Biazotto, Daniel Feitosa, Paris Avgeriou, et al.: *Automating Technical Debt Management: Insights from Practitioner Discussions in Stack Exchange*. arXiv:2502.03153 (2025). <https://arxiv.org/abs/2502.03153>
- [22] Leonhard Applis, Yuntong Zhang, Shanchao Liang, et al.: *Unified Software Engineering Agent as AI Software Engineer*. arXiv:2506.14683 (2025). <https://arxiv.org/abs/2506.14683>
- [23] Islem Bouzenia, Michael Pradel: *Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories*. arXiv:2506.18824 (2025). <https://arxiv.org/abs/2506.18824>
- [24] Jiayi Kuang, Yinghui Li, Xin Zhang, et al.: *Process-Level Trajectory Evaluation for Environment Configuration in Software Engineering Agents*. arXiv:2510.25694 (2025). <https://arxiv.org/abs/2510.25694>
- [25] Quentin Herr: *Landauer-Limited Dissipation in Quantum-Flux-Parametron Logic*. arXiv:2504.04284 (2025). <https://arxiv.org/abs/2504.04284>
- [26] I. Neri, M. Lopez-Suarez, D. Chiuchiu, et al.: *Reset and switch protocols at Landauer limit in a graphene buckled ribbon*. arXiv:1507.07450 (2015). <https://arxiv.org/abs/1507.07450>